Assignment - 4

Project Title : Student Performance Tracker

main.py

```
import psycopg2
import time
import sys
import os

# --- 5. SPINNER FUNCTIONS (Formatted to your requirements) ---
def saving_data_spinner(duration=2):
    spinner = ['|', '/', '-', '\\']
    end_time = time.time() + duration
    idx = 0
    print("Saving data... ", end="")
    while time.time() < end_time:
        sys.stdout.write(f"\rSaving data... {spinner[idx % len(spinner)]}")
        sys.stdout.flush()
        idx += 1
        time.sleep(0.1)
    sys.stdout.write("\rData Saved Successfully! \n")

def reloading_data_spinner(duration=2):
    spinner = ['|', '/', '-', '\\']
    end_time = time.time() + duration
    idx = 0
    print("Reloading program... ", end="")
    while time.time() < end_time:
        sys.stdout.write(f"\rReloading program... {spinner[idx % len(spinner)]}")
        sys.stdout.flush()
        idx += 1
        time.sleep(0.1)
    sys.stdout.write("\r Reloading Succesfull \n")

def run_spinner(duration=2):
    spinner = ['|', '/', '-', '\\']
    end_time = time.time() + duration
    idx = 0
    print("Loading data... ", end="")
    while time.time() < end_time:
        sys.stdout.write(f"\rLoading data... {spinner[idx % len(spinner)]}")
        sys.stdout.flush()
        idx += 1
        time.sleep(0.1)
    sys.stdout.write("\rData Loaded Successfully! \n")


# --- 2. OBJECT-ORIENTED DESIGN & 6. DATABASE INTEGRATION ---
class Student:
    def __init__(self, name, roll_number, math=None, science=None, english=None):
        self.name = name.upper()
        self.roll_number = roll_number
        self.grades = {"Math": math, "Science": science, "English": english}

    def add_grades(self, math, science, english):
        self.grades["Math"] = math
        self.grades["Science"] = science
        self.grades["English"] = english

    def calculate_average(self):
        scores = [v for v in self.grades.values() if v is not None]
        if not scores: return 0.0
        return round(sum(scores) / len(scores), 2)

    def display_info(self):
        print('-' * 85)
        print(f"| {'SNO/ROLL:':<10} {self.roll_number:<15} | {'NAME:':<6} {self.name:<43} |")
        print('-' * 85)
        m = self.grades['Math'] if self.grades['Math'] is not None else 'N/A'
        s = self.grades['Science'] if self.grades['Science'] is not None else 'N/A'
        e = self.grades['English'] if self.grades['English'] is not None else 'N/A'
        print(f"| GRADES -> Math: {m:<10} Science: {s:<10} English: {e:<10}            |")
        print(f"| AVERAGE: {self.calculate_average():<71} |")
        print('-' * 85)

class StudentTracker:
    def __init__(self):
        # 6. PostgreSQL Database
        EXTERNAL_URL = "postgresql://edutrack_db_qufk_user:WRnGZGxftYOAmNaG0uHTrc8Sgc6RdFmK@dpg-d9g9t9mrnols73c4lovg-a.singapore-postgres.render.com/edutrack_db_qufk"
        self.conn = psycopg2.connect(EXTERNAL_URL)
        self.cursor = self.conn.cursor()
        self.cursor.execute('''CREATE TABLE IF NOT EXISTS students
                               (roll_number TEXT PRIMARY KEY, name TEXT, math REAL, science REAL, english REAL)''')
        self.conn.commit()

    # 1. Core Functionalities
    def add_student(self, name, roll_number):
        try:
            self.cursor.execute("INSERT INTO students (roll_number, name) VALUES (%s, %s)", (roll_number, name))
            self.conn.commit()
            return "Student Added Successfully"
        except psycopg2.IntegrityError:
            return "ERROR: STUDENT ALREADY EXISTS"

    def add_grades(self, roll_number, math, science, english):
        self.cursor.execute("SELECT * FROM students WHERE roll_number=%s", (roll_number,))
        if not self.cursor.fetchone():
            return "ERROR: STUDENT NOT FOUND"
        self.cursor.execute("UPDATE students SET math=%s, science=%s, english=%s WHERE roll_number=%s     ", (math, science, english, roll_number))
        self.conn.commit()
        return "Grades Assigned Successfully"

    def get_all_students(self):
        self.cursor.execute("SELECT roll_number, name, math, science, english FROM students")
        return [Student(row[1], row[0], row[2], row[3], row[4]) for row in self.cursor.fetchall()]

    # 9. BONUS FEATURES
    def subject_topper(self, subject):
        self.cursor.execute(f"SELECT name, {subject.lower()} FROM students WHERE {subject.lower()} IS NOT NULL ORDER BY {subject.lower()} DESC LIMIT 1")
        row = self.cursor.fetchone()
        if row: return f"Topper in {subject}: {row[0]} with {row[1]} marks"
        return f"No data for {subject}"

    def class_average(self, subject):
```

```
        self.cursor.execute(f"SELECT AVG({subject.lower()}) FROM students WHERE {subject.lower()} IS NOT NULL")
        row = self.cursor.fetchone()
        if row[0]: return f"Class Average for {subject}: {round(row[0], 2)}"
        return f"No data for {subject}"

    def backup_data(self):
        students = self.get_all_students()
        with open("student_backup.txt", "w") as f:
            for s in students:
                f.write(f"{s.roll_number},{s.name},{s.grades['Math']},{s.grades['Science']},{s.grades['English']}\n")
        return "Data backed up to student_backup.txt"

    # --- 5. LOOP FOR CONTINUOUS INTERACTION ---

def main():
    tracker = StudentTracker()
    students = tracker.get_all_students()

    if not students:
        print('\n>>> NO STUDENTS ENROLLED')
    else:
        print('EXISTING STUDENTS:\n')
        for s in students:
            s.display_info()

    while True:
        print("\n1. Add Student")
        print("2. Assign Grades")
        print("3. View Subject Topper (Bonus)")
        print("4. View Class Average (Bonus)")
        print("5. Save Data Locally (Bonus)")
        print("6. Reload")
        print("7. Exit")

        try:
            choice = int(input("\nChoose an option: "))
        except ValueError:
            print('\n\n !!!!!!!!! INVALID INPUT')
            continue

        if choice == 1:
            print('\nFill The Data asked below:\n')
            name = input('Enter Student Name: ')
            roll = input('Enter Roll Number: ')
            print('\n', tracker.add_student(name, roll))

        elif choice == 2:
            roll = input('Enter Roll Number: ')
            try:
                # 4. Conditional Logic and Flow Control (0 to 100 validation)
                m = float(input("Math Grade (0-100): "))
                s = float(input("Science Grade (0-100): "))
                e = float(input("English Grade (0-100): "))

                if all(0 <= val <= 100 for val in [m, s, e]):
                    print('\n', tracker.add_grades(roll, m, s, e))
                else:
                    print('\nERROR: Grades must be between 0 and 100')
            except ValueError:
                print('\nERROR: Invalid Number')

        elif choice == 3:
            sub = input("Enter Subject (Math/Science/English): ")
            if sub.lower() in ['math', 'science', 'english']:
                print('\n', tracker.subject_topper(sub))
            else:
                print('\n Invalid Subject')

        elif choice == 4:
            sub = input("Enter Subject (Math/Science/English): ")
            if sub.lower() in ['math', 'science', 'english']:
                print('\n', tracker.class_average(sub))
            else:
                print('\n Invalid Subject')

        elif choice == 5:
            print('\n', tracker.backup_data())

        elif choice == 6:
            print('\n')
            reloading_data_spinner(2)
            print('\nStudent Performance Tracker\n')
            return main()

        elif choice == 7:
            saving_data_spinner(2)
            print('\nThank you for using >>> STUDENT TRACKER <<<\n\n')
            break
        else:
            print('\n\n !!!!!!!!! INVALID INPUT')

if __name__ == '__main__':
    run_spinner(2)
    print('\nStudent Performance Tracker\n')
    main()
```

app.py

```
from flask import Flask, request, render_template_string, redirect import
psycopg2

app = Flask(__name__)

HTML_TEMPLATE = """
<!DOCTYPE html>
<html>
<head>
        <title>Student Tracker</title>
        <style>
        body { font-family: Arial; padding: 20px; }
        table { border-collapse: collapse; width: 80%; margin-top: 20px; } th,
        td { border: 1px solid black; padding: 8px; text-align: left; } th {
        background-color: #f2f2f2; }
        </style>
</head>
<body>
        <h2>Student Performance Tracker (Web Interface)</h2>
        <form method="POST" action="/add">
        <input type="text" name="roll" placeholder="Roll Number" required>
        <input type="text" name="name" placeholder="Student Name" required>
        <button type="submit">Add Student</button>
        </form>
```

```
        <table>
            <tr>
            <th>Roll Number</th>
            <th>Name</th>
            <th>Math</th>
            <th>Science</th>
            <th>English</th>
        </tr>
        {% for s in students %}
        <tr>
            <td>{{ s[0] }}</td>
            <td>{{ s[1] }}</td>
            <td>{{ s[2] if s[2] != None else 'N/A' }}</td>
            <td>{{ s[3] if s[3] != None else 'N/A' }}</td>
            <td>{{ s[4] if s[4] != None else 'N/A' }}</td>
        </tr>
        {% endfor %}
        </table>
</body>
</html> """

@app.route('/') def
index():
        INTERNAL_URL = "postgresql://edutrack_db_qufk_user:WRnGZGxftYOAmNaG0uHTrc8Sgc6RdFmK@dpg-d9g9t9mrnols73c4lovg-a.singapore-postgres.render.com/edutrack_db_qufk"
        conn = psycopg2.connect(INTERNAL_URL)
        cur = conn.cursor()
        cur.execute("SELECT roll_number, name, math, science, english FROM students")
        students = cur.fetchall()
        conn.close()
        return render_template_string(HTML_TEMPLATE, students=students)

@app.route('/add', methods=['POST']) def
add():
        INTERNAL_URL = "postgresql://edutrack_db_qufk_user:WRnGZGxftYOAmNaG0uHTrc8Sgc6RdFmK@dpg-d9g9t9mrnols73c4lovg-a/edutrack_db_qufk"
        conn = psycopg2.connect(INTERNAL_URL)
        cur = conn.cursor()
        try:
        cur.execute("INSERT INTO students (roll_number, name) VALUES (%s, %s)",
                    (request.form['roll'], request.form['name']))
        conn.commit()
    except psycopg2.IntegrityError: pass
        conn.close()
        return redirect('/')

    if __name__ == "__main__":
        app.run(debug=True)
```

requirements.txt

```
Flask==3.0.3
gunicorn==22.0.0
psycopg2-binary
```

Procfile

```
web: gunicorn app:app
```

User_Guide.txt

Student Performance Tracker(EduTrack) - User Guide

OVERVIEW
This application allows teachers to persistently track student grades across Math, Science, and English using both a Command Line Interface (CLI) and a web-based Flask Dashboard.

SETUP & INSTALLATION
1. Ensure Python 3.x is installed on your system.
2. Install the necessary web dependencies by running:
  pip install -r requirements.txt
3. The database (`students.db`) will be generated automatically upon your first launch.

USING THE CLI (main.py)
1. Run the script: `python main.py`
2. Follow the on-screen spinner to access the main menu.
3. Choose Option 1 to register a new student (requires Name and Unique Roll Number).
4. Choose Option 2 to assign grades. You will be prompted to enter a grade (0-100) for Math, Science, and English.
5. Choose Option 5 to execute the backup bonus feature, exporting all data to `student_backup.txt`.

USING THE WEB APP (app.py)
1. Run the script: `python app.py`
2. Open your web browser and navigate to http://127.0.0.1:5000
3. The dashboard will display all registered students and their grades synced directly from the SQLite database.
4. You can quickly add new students using the form at the top of the webpage.

DEPLOYMENT INSTRUCTIONS (RENDER)
(Note: Due to encountering account verification issues with Heroku during setup, Render was utilized as the cloud hosting service for this deployment.)
1. Push your local project code to a public GitHub repository.
2. Log into Render.com, click "New +", and select "Web Service".
3. Connect your GitHub account and select your project repository.
4. Set the Build Command to `pip install -r requirements.txt` and the Start Command to `gunicorn app:app`.
5. Click "Create Web Service" and wait for the deployment to finish to get your live URL.

Deployed Link: https://edutrack-1-r3mx.onrender.com/

Student backup.txt

101,AMAR KUMAR,88.0,75.0,91.0
102,RAJKUMAR SAHU,86.0,67.0,85.0
103,SOPHIYA KHATUN,91.0,78.0,79.0
104,STEVE JOHNSON ,89.0,95.0,79.0

User Interface

```
Student Performance Tracker

EXISTING STUDENTS:

--------------------------------------------------------------------------------
| SNO/ROLL:  101             | NAME:  AMAR KUMAR                               |
--------------------------------------------------------------------------------
| GRADES -> Math: 88.0       Science: 75.0       English: 91.0                 |
| AVERAGE: 84.67                                                               |
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
| SNO/ROLL:  102             | NAME:  RAJKUMAR SAHU                            |
--------------------------------------------------------------------------------
| GRADES -> Math: 86.0       Science: 67.0       English: 85.0                 |
| AVERAGE: 79.33                                                               |
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
| SNO/ROLL:  103             | NAME:  SOPHIYA KHATUN                           |
--------------------------------------------------------------------------------
| GRADES -> Math: 91.0       Science: 78.0       English: 79.0                 |
| AVERAGE: 82.67                                                               |
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
| SNO/ROLL:  104             | NAME:  STEVE JOHNSON                            |
--------------------------------------------------------------------------------
| GRADES -> Math: 89.0       Science: 95.0       English: 79.0                 |
| AVERAGE: 87.67                                                               |
--------------------------------------------------------------------------------

1. Add Student
2. Assign Grades
3. View Subject Topper (Bonus)
4. View Class Average (Bonus)
5. Save Data Locally (Bonus)
6. Reload
7. Exit

Choose an option:
```